

Skip List
–Destroy List
–Program Analysis

Program:

```
void destroyList(struct SkipList *lst) {
    // Start at the header
    struct Node *current = lst->header;
    struct Node *next;

    // We only need to traverse level 0 (the "local train" level)
    // because it is the only level guaranteed to contain EVERY node.
    while (current != NULL) {
        // 1. Save the pointer to the next node
        next = current->forward[0];

        // 2. Free the dynamically allocated array of pointers
        free(current->forward);

        // 3. Free the node itself
        free(current);

        // 4. Move to the next node
        current = next;
    }

    // Finally, free the main Skip List structure
    free(lst);
    cout << "Memory successfully freed. Skip List destroyed." << endl;
}

...
case 5:
    cout << "Destroying the current list..." << endl;
    destroyList(list);
    // CRITICAL: We must give the program a fresh, empty list
    // so it doesn't crash if the user tries to insert again.
    break;
```

Program Analysis

```
destroyList(list);
```

destroyList(list: ff08) → passed .

And,

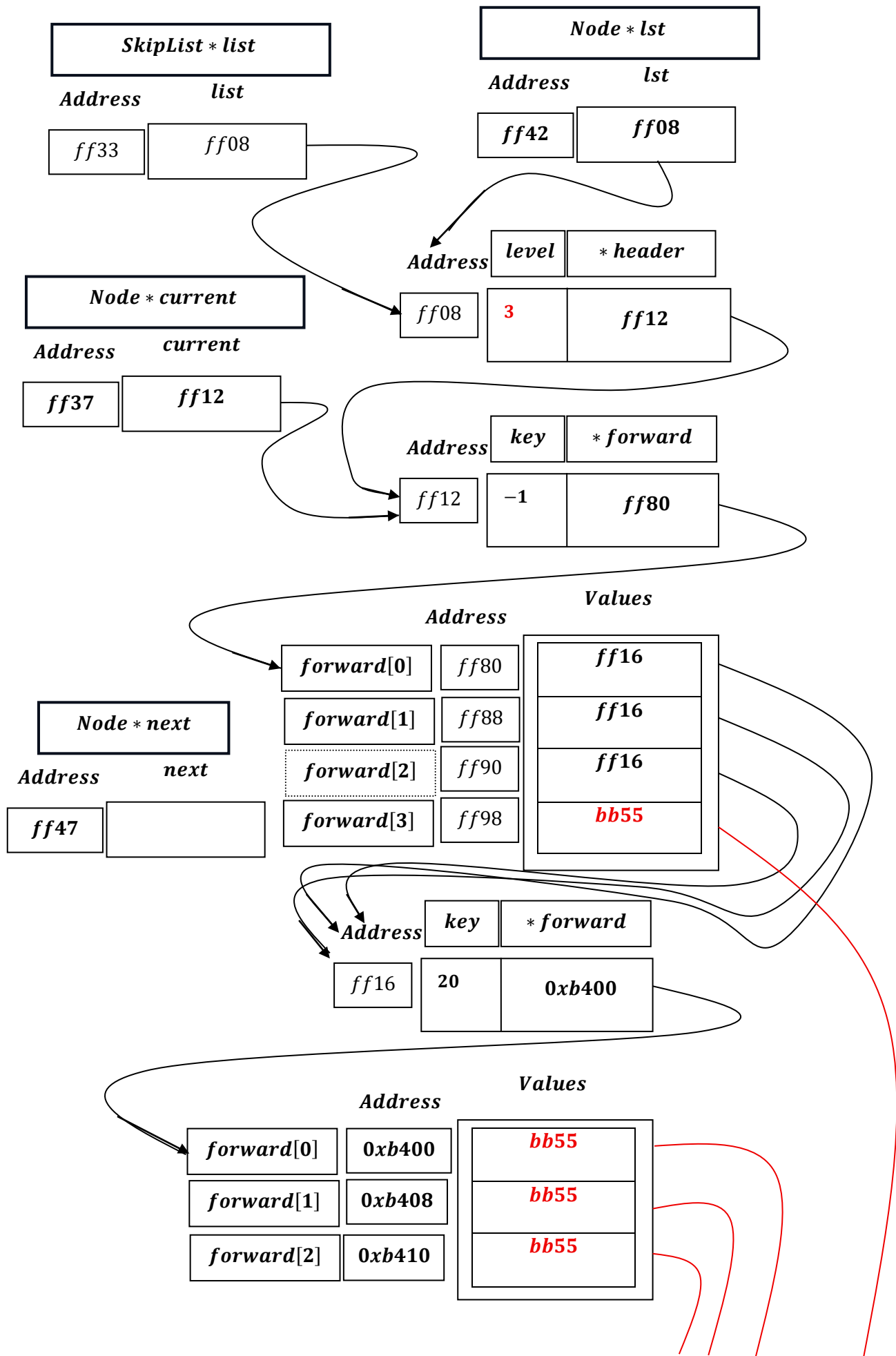
```
void destroyList(struct SkipList *lst)
```

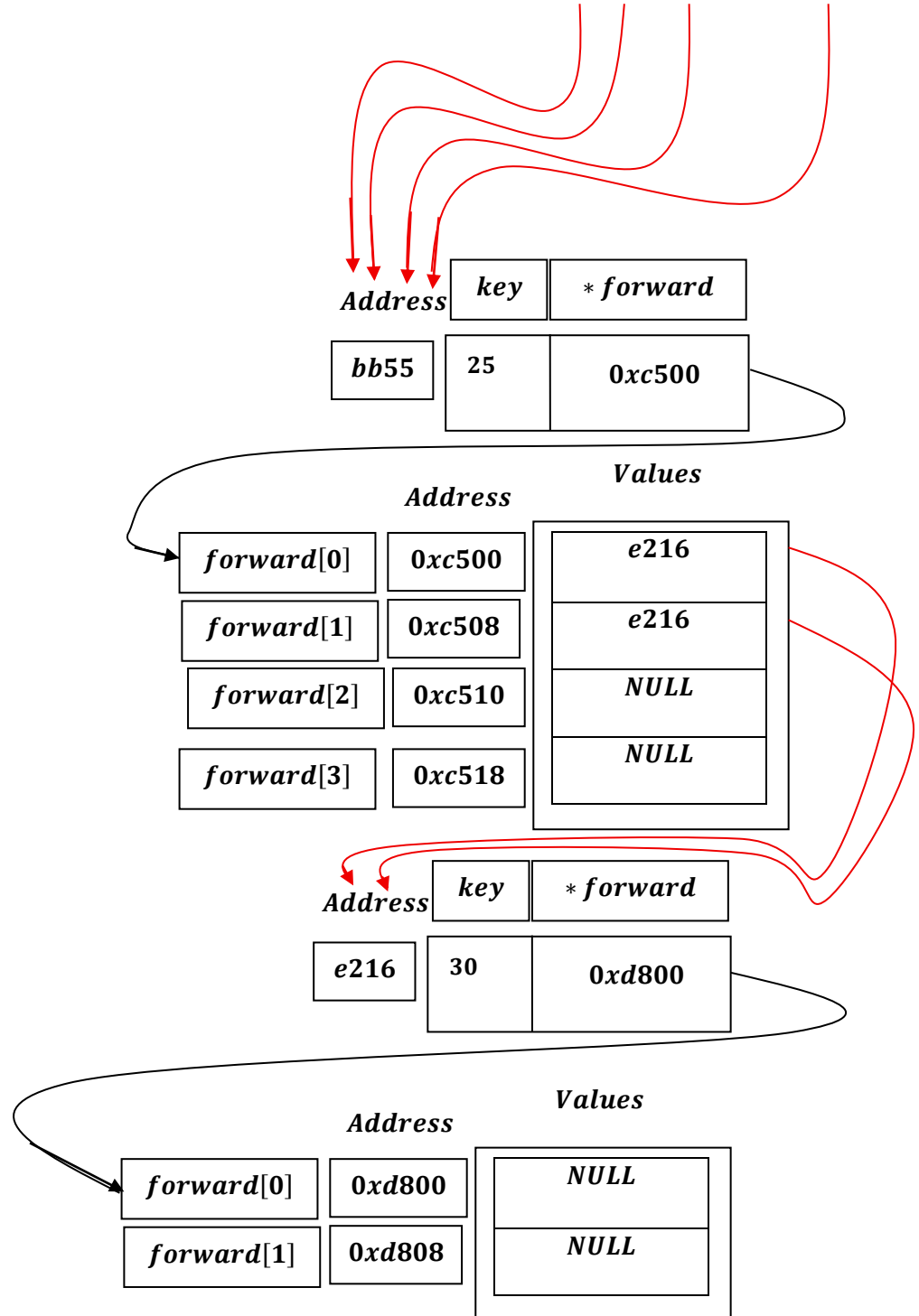
destroyList(list := lst: ff08) → Assigned

```
struct Node *current = lst->header;
```

current = lst → header = ff12.

```
struct Node *next;
```





```
while (current != NULL) {
    // 1. Save the pointer to the next node
    next = current->forward[0];

    // 2. Free the dynamically allocated array of pointers
    free(current->forward);
    // 3. Free the node itself
    free(current);

    // 4. Move to the next node
    current = next;
}
```

While(current: ff12 $\neq$ NULL)[True] :

next = current: ff12 $\rightarrow$ forward[0]: ff16

free(current: ff12 $\rightarrow$ forward)

*The pointer **current** $\rightarrow$ **forward** holds the starting address of this chunk, which is **ff80**. This is where **forward[0]** lives.*

***The Offset:** The address **ff88** is simply the location of the second element in that array (**forward[1]**). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.*

*When the program executes **free(current $\rightarrow$ forward)**, it tells the system to deallocate the entire chunk of memory that begins at **ff80**. Because **ff88** is just a part of that continuous chunk, it gets completely freed at the exact same time.*

It will not add any offsets and delete each block of memory but all at once.

```
free(current);
```

free(current: ff12)

```
current = next;
```

current = next = ff16 .

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Address level * header*

ff08 3 ff12

Address Values

ff12

*Node * next*

Address next

ff47 ff16

*Node * current*

Address current

ff37 ff16

*Address key * forward*

ff16 20 0xb400

Values

Address

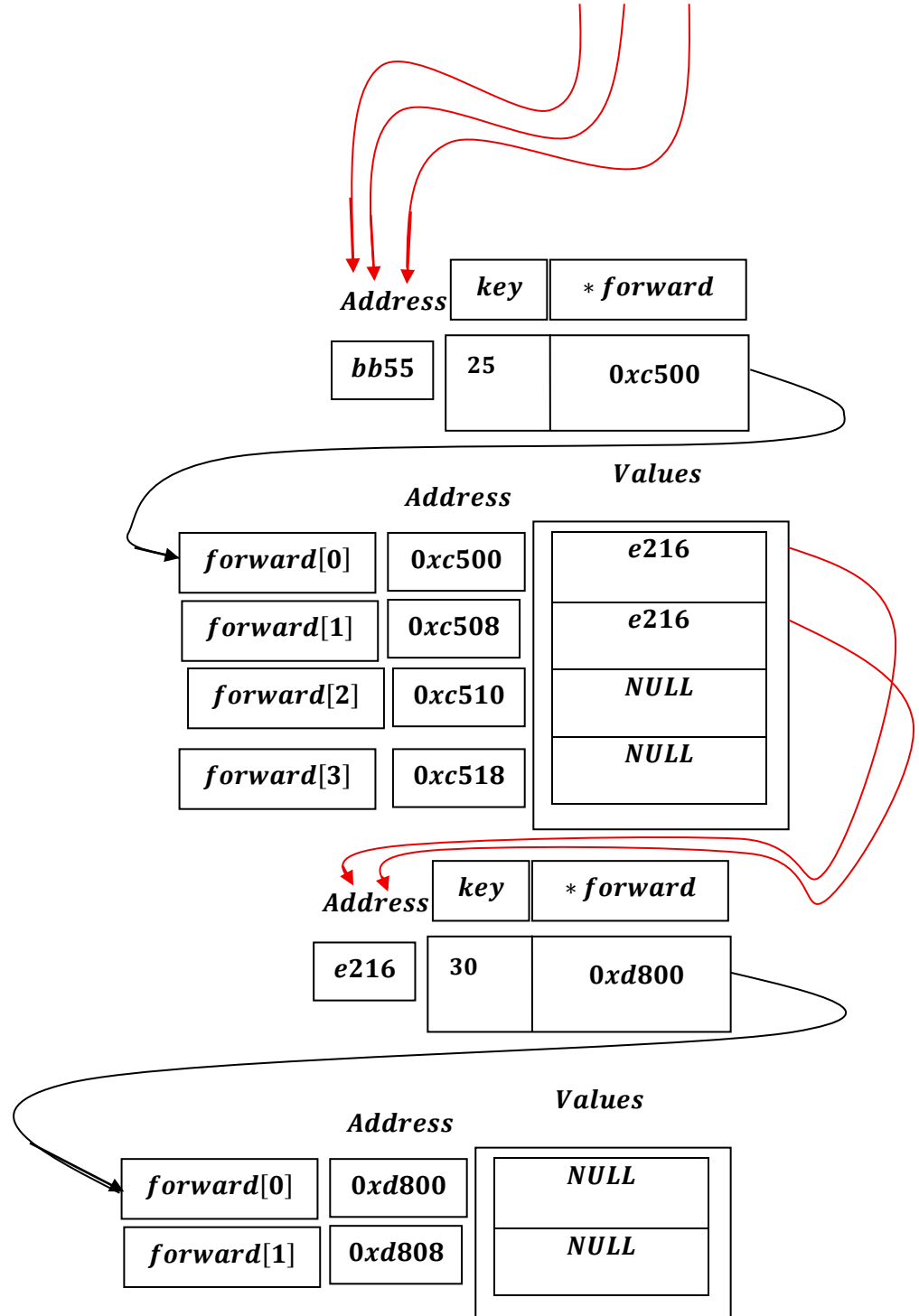
ff80
ff88
ff90
ff98

Values

Address

forward[0] 0xb400
forward[1] 0xb408
forward[2] 0xb410

bb55
bb55
bb55



While(current: ff16 $\neq$ NULL)[True] :

next = current: ff16 $\rightarrow$ forward[0]: bb55

free(current: ff16 $\rightarrow$ forward)

The pointer **current** $\rightarrow$ **forward** holds the starting address of this chunk, which is **0xb400**. This is where **forward[0]** lives.

The Offset: The address **0xb408** is simply the location of the second element

in that array (*forward[1]*). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.

When the program executes *free(current → forward)*, it tells the system to deallocate the entire chunk of memory that begins at *0xb400*. Because *0xb408* is just a part of that continuous chunk, it gets completely freed at the exact same time.

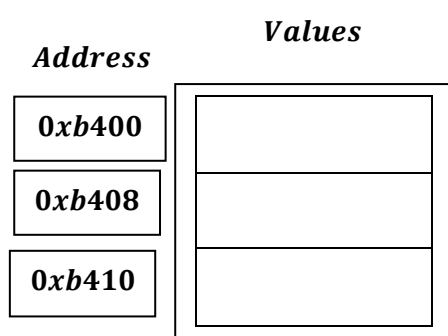
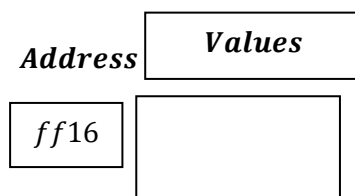
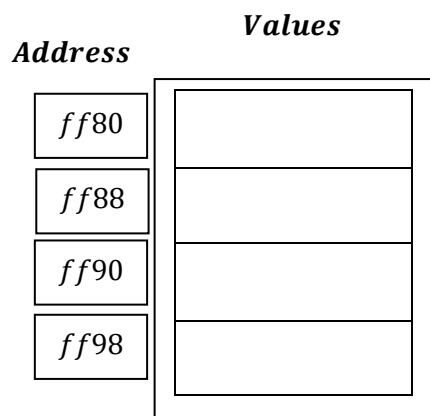
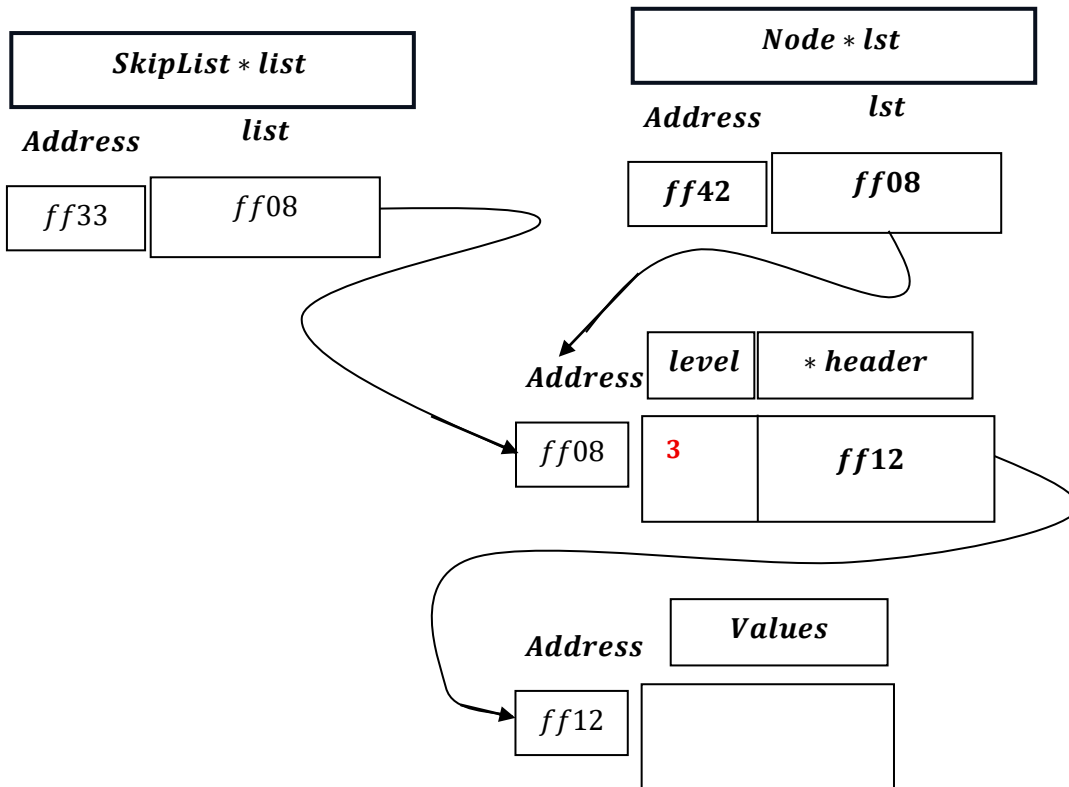
It will not add any offsets and delete each block of memory but all at once.

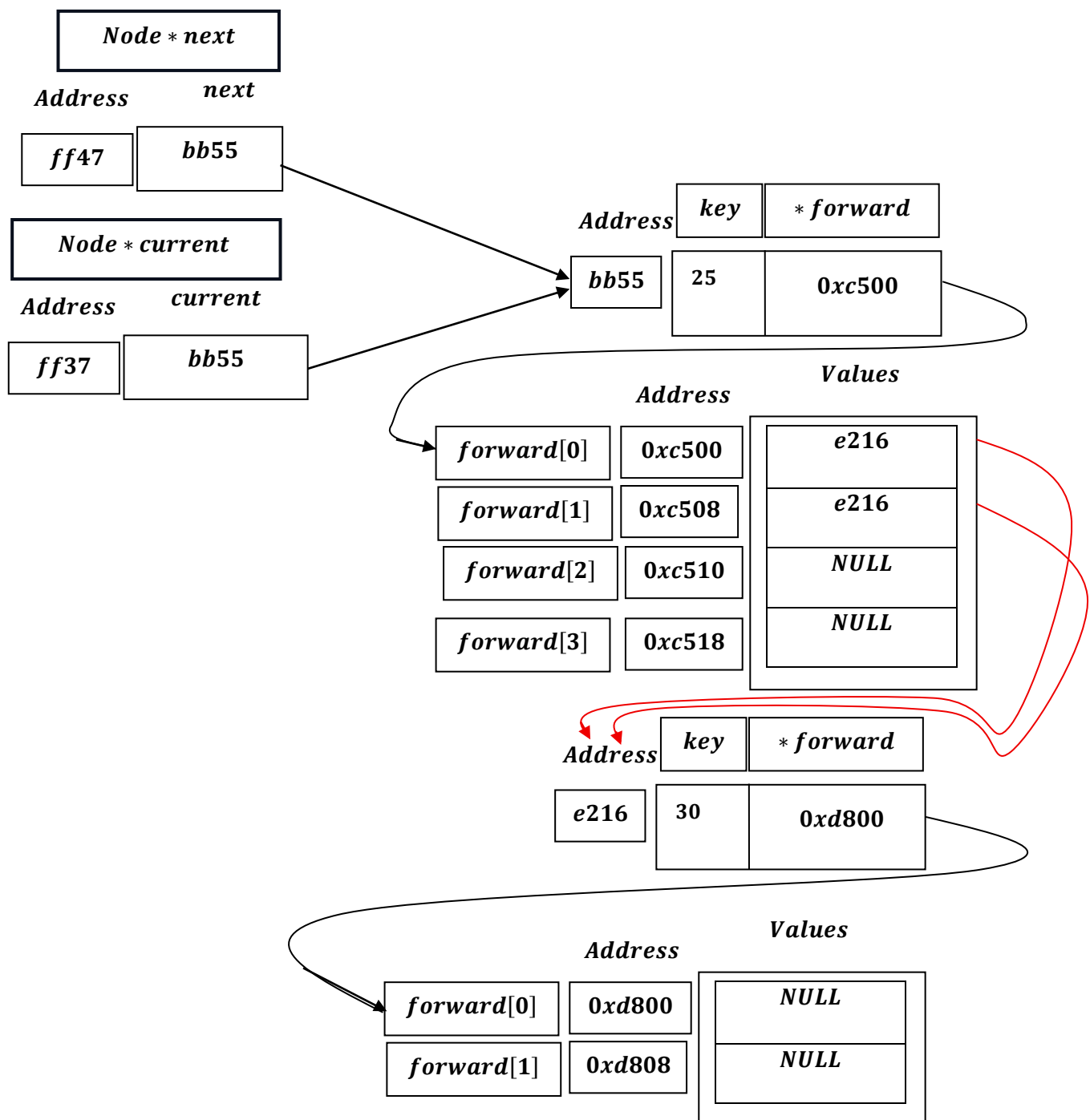
```
free(current);
```

free(current: ff16)

```
current = next;
```

current = next = bb55 .





While(current: bb55 $\neq$ NULL)[True] :

next = current: bb55 $\rightarrow$ forward[0]: e216

free(current: bb55 $\rightarrow$ forward)

The pointer **current $\rightarrow$ forward** holds the starting address of this chunk, which is **0xc500**. This is where **forward[0]** lives.

The Offset: The address **0xc508** is simply the location of the second element in that array (**forward[1]**). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.

When the program executes **free(current → forward)**, it tells the system to deallocate the entire chunk of memory that begins at **0xc500**. Because **0xc508** is just a part of that continuous chunk, it gets completely freed at the exact same time.

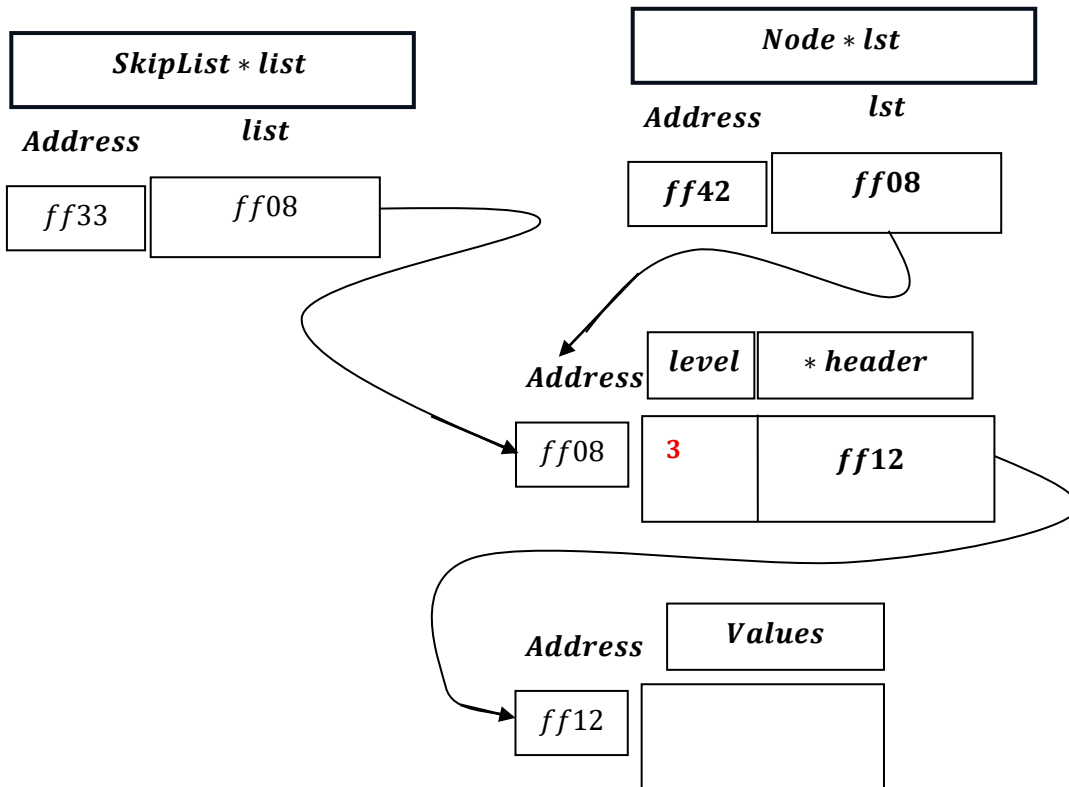
It will not add any offsets and delete each block of memory but all at once.

```
free(current);
```

free(current: bb55)

```
current = next;
```

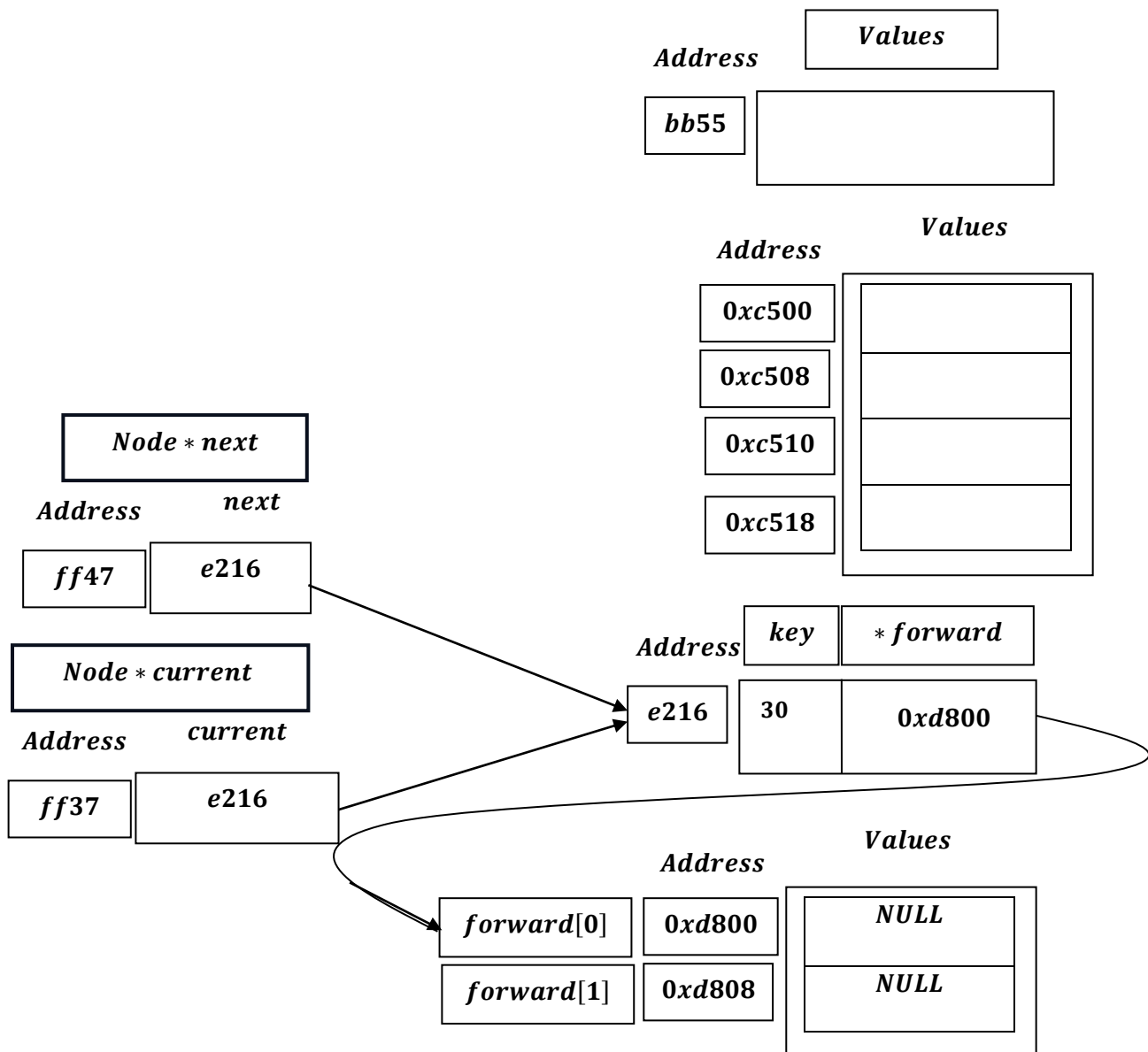
current = next = e216 .



Address	Values
ff80	
ff88	
ff90	
ff98	

Address	Values
ff16	

Address	Values
0xb400	
0xb408	
0xb410	



While(current: e216 $\neq$ NULL)[True] :

next = current: e216 $\rightarrow$ forward[0]: NULL

free(current: e216 $\rightarrow$ forward)

The pointer **current $\rightarrow$ forward** holds the starting address of this chunk, which is **0xd800**. This is where **forward[0]** lives.

***The Offset:** The address **0xd808** is simply the location of the second element in that array (**forward[1]**). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.*

*When the program executes **free(current → forward)**, it tells the system to deallocate the entire chunk of memory that begins at **0xd800**. Because **0xd808** is just a part of that continuous chunk, it gets completely freed at the exact same time.*

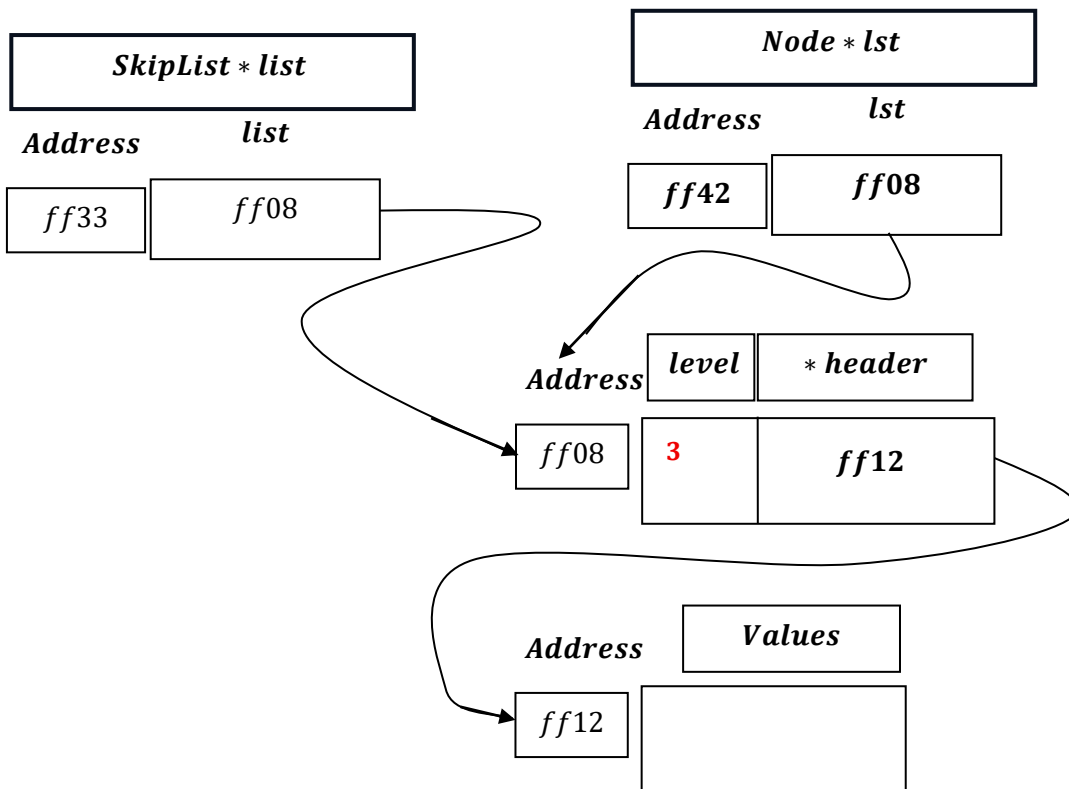
It will not add any offsets and delete each block of memory but all at once.

```
free(current);
```

free(current: e216)

```
current = next;
```

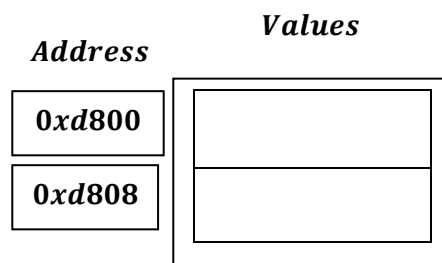
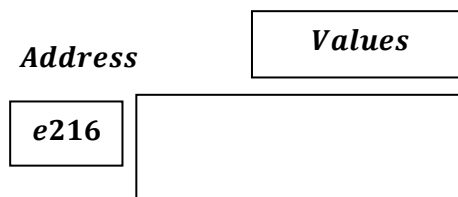
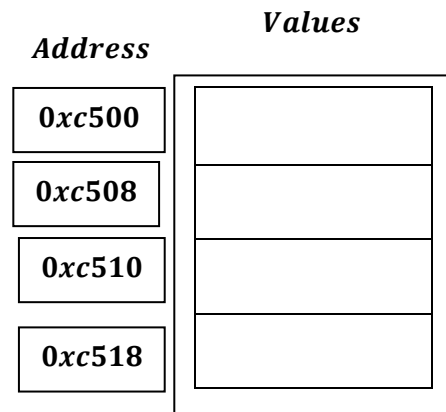
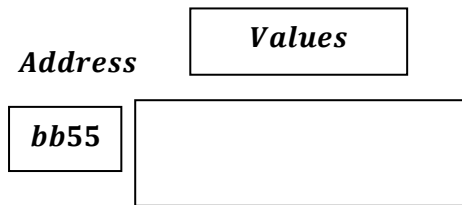
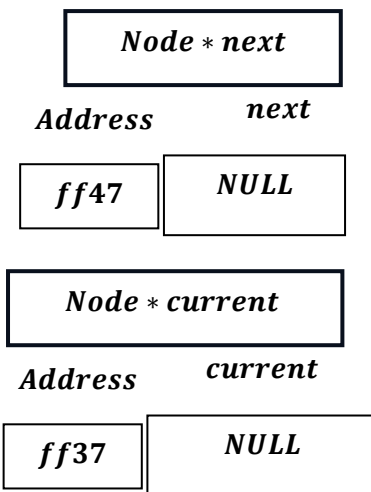
current = next = NULL .



Address	Values
ff80	
ff88	
ff90	
ff98	

Address	Values
ff16	

Address	Values
0xb400	
0xb408	
0xb410	



*While(current: NULL ≠ NULL) [False,
Loop
Exits]*

Now we execute the final freeing:

```
free(lst);
```

free(lst: ff08)

SkipList * list

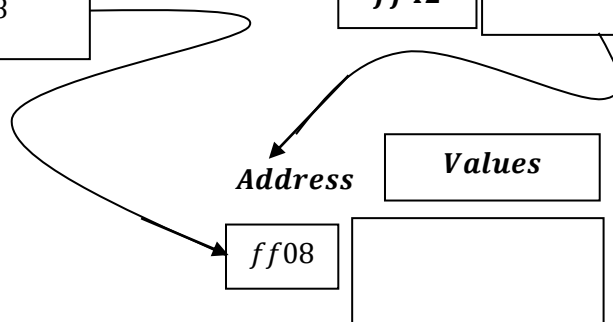
Address list

ff33 ff08

Node * lst

Address lst

ff42 ff08



Address Values
ff08

Address Values
ff12

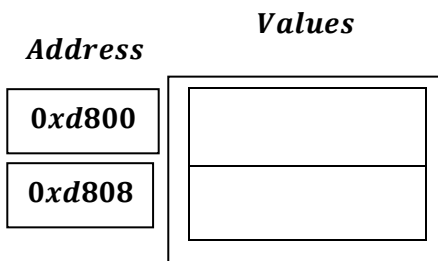
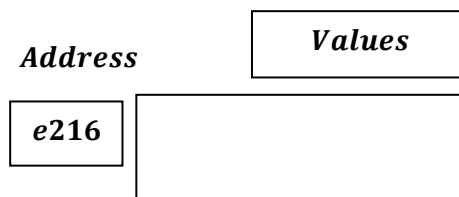
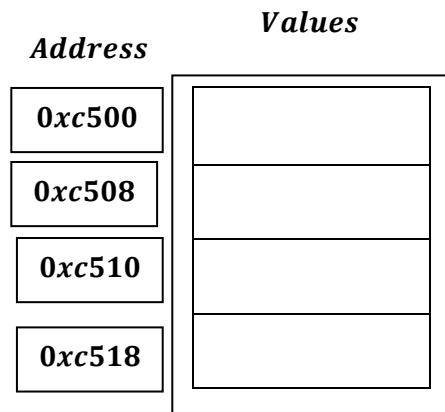
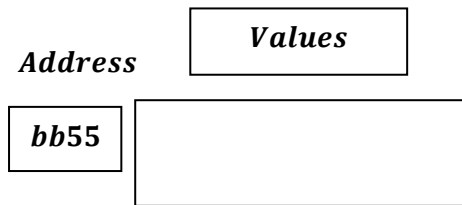
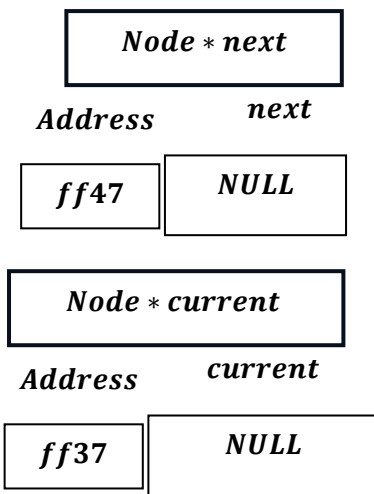
Address Values

ff80	
ff88	
ff90	
ff98	

Address Values
ff16

Address Values

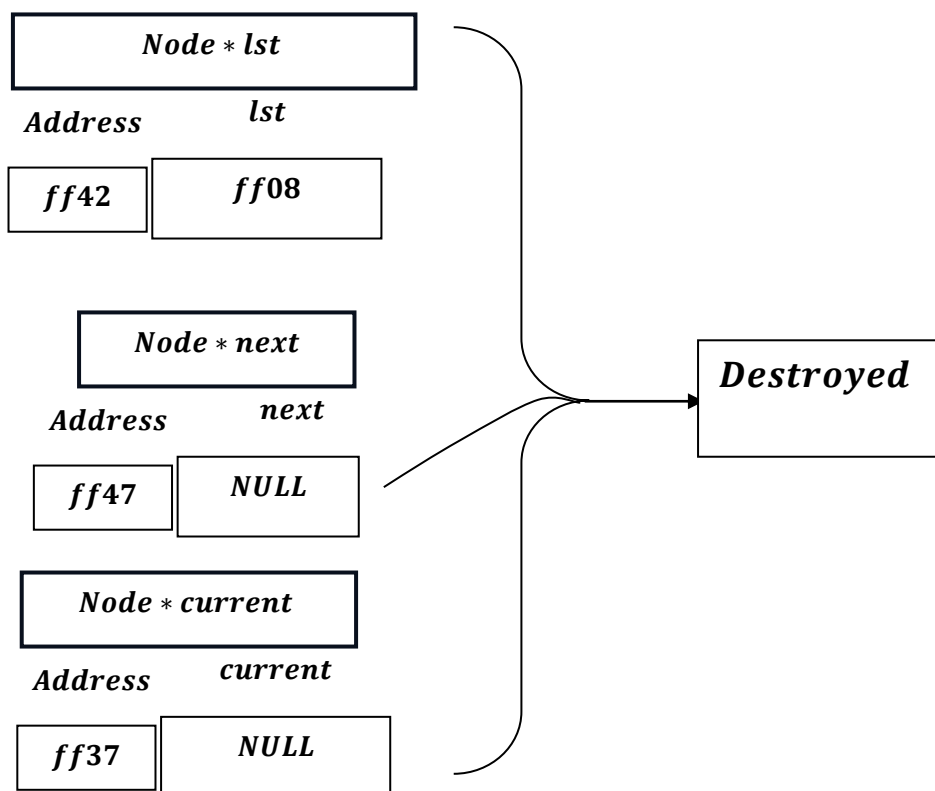
0xb400	
0xb408	
0xb410	



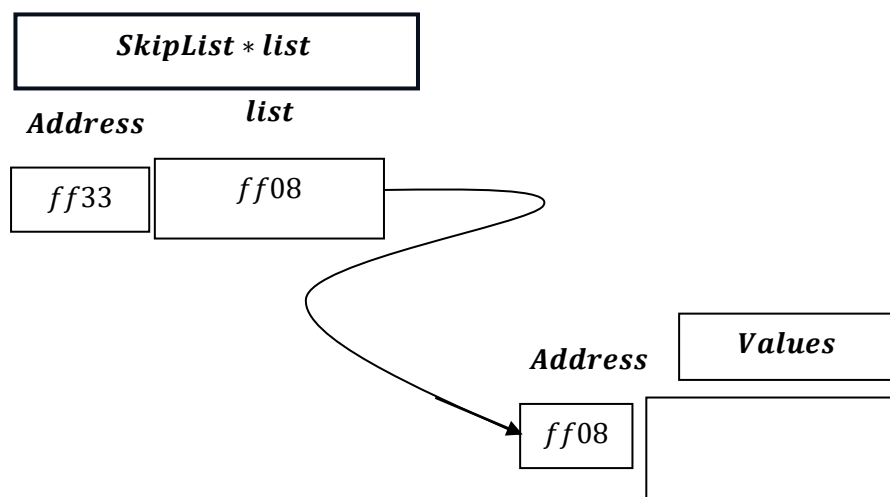
And, then it prints:

```
cout << "Memory successfully freed. Skip List destroyed." << endl;
```

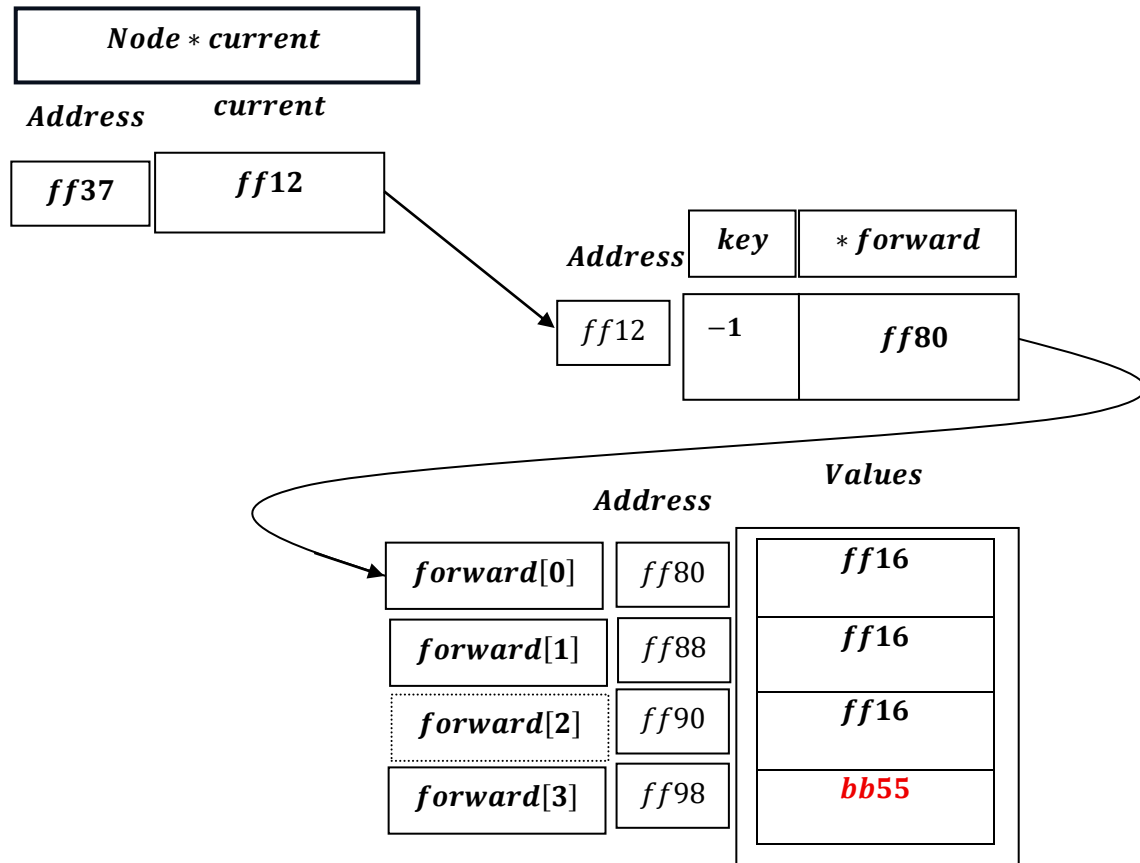
And , as function ends.



And we are left with:



Note how we are freeing the elements:

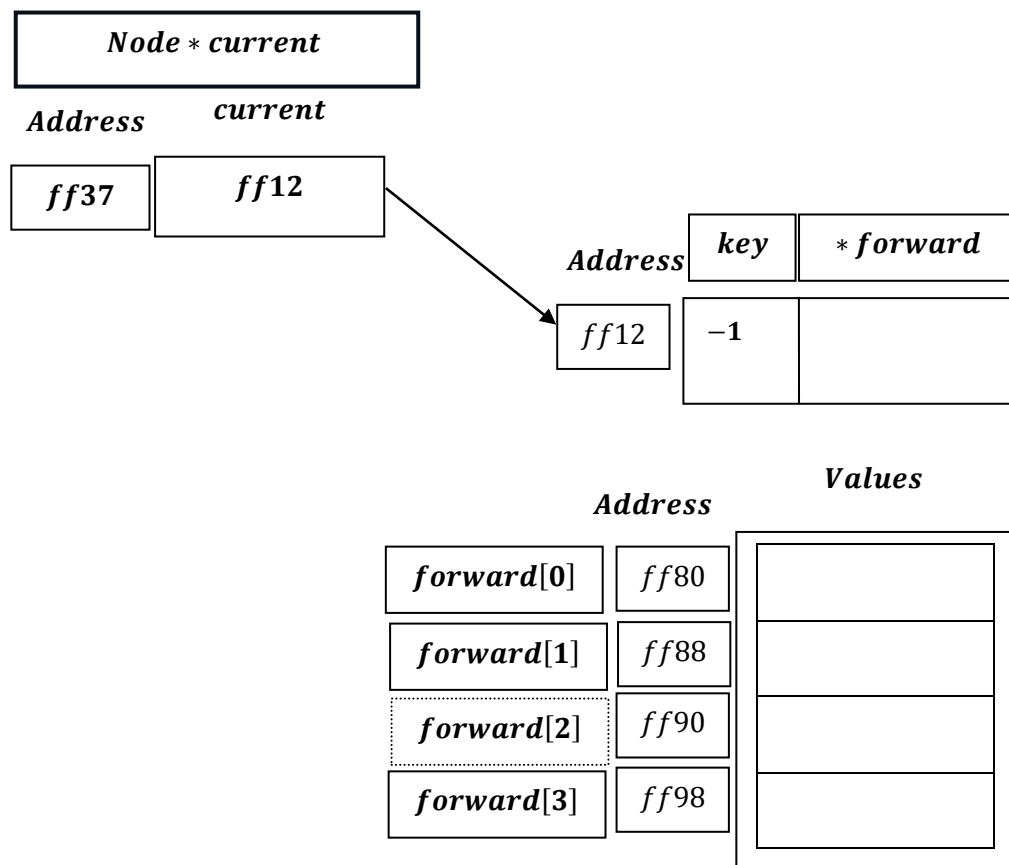


First current → forward is freed

```
free(current->forward);
```

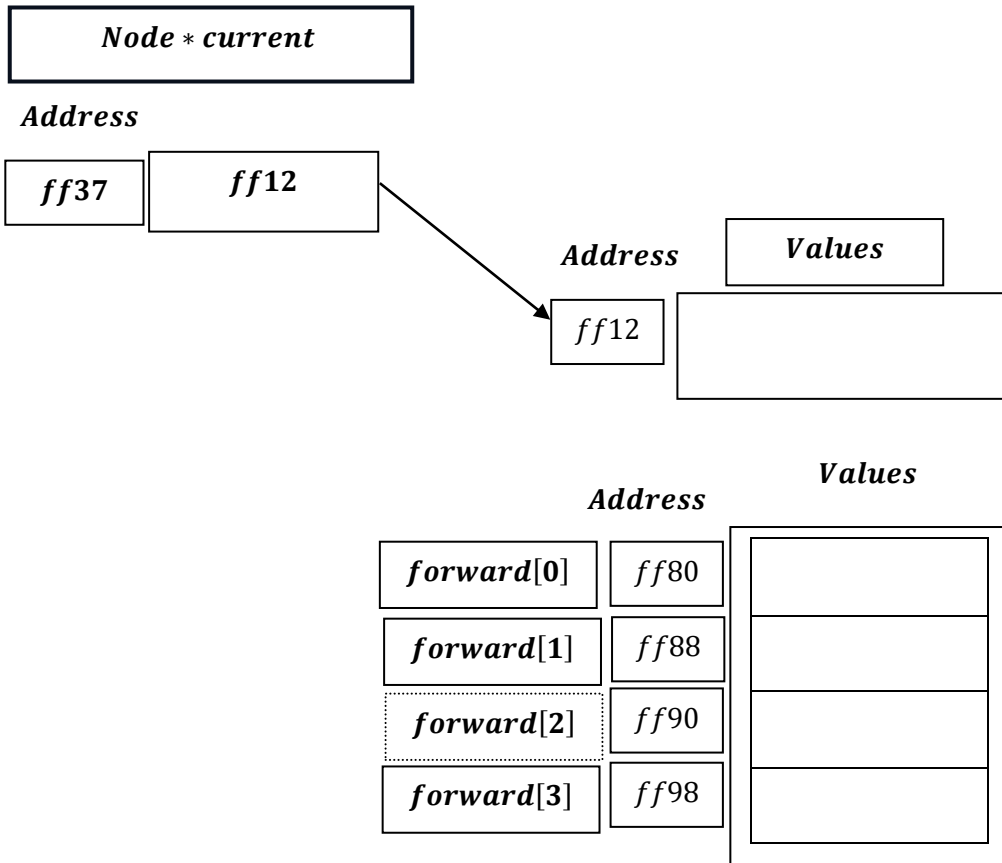
The pointer **current** → **forward** holds the starting address of this chunk, which is **ff80**. And next addresses [**ff88**, **ff90**, **ff98**] automatically gets freed all at once.

forward is dynamically allocated as a single contiguous array. The elements are contiguous, with an **8 – byte** stride between the starting addresses of consecutive elements because each element stores an **8 – byte pointer**. Therefore, **free(current → forward)** deallocates the entire array allocation in one operation.



Next , **current** is freed , this wipes out value of **Key**.

```
free(current);
```



So, there are two way freeing of memory.
